ETH zürich

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CachePool: Many-core cluster of customizable, lightweight scalar-vector PEs for irregular L2 data-plane workloads

Integrated Systems Laboratory (ETH Zürich)

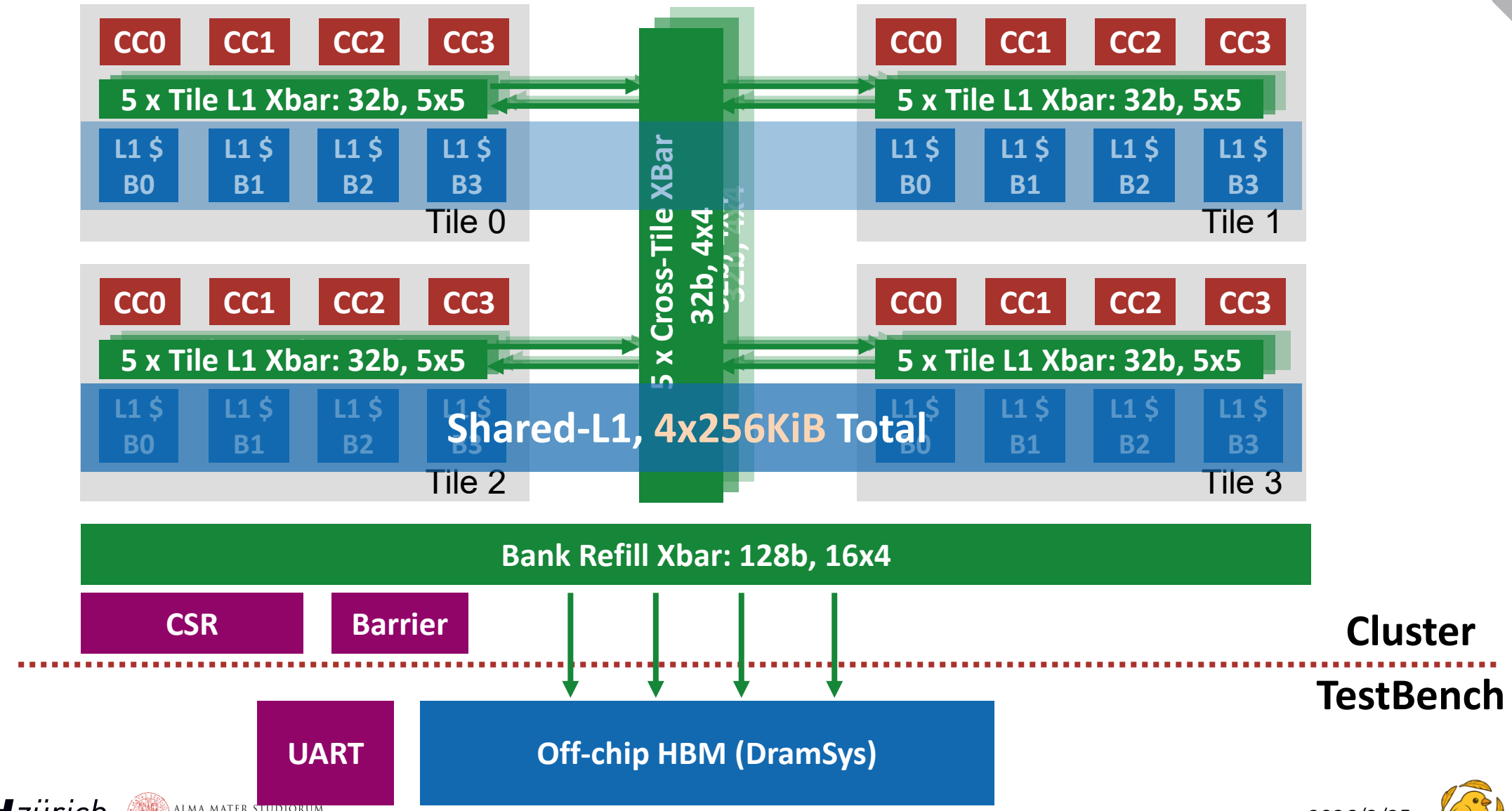
Zexin Fu, Diyou Shen zexifu, dishen@iis.ee.ethz.ch

Alessandro Vanelli-Coralli avanelli@iis.ee.ethz.ch
Luca Benini lbenini@iis.ee.ethz.ch

PULP Platform
Open Source Hardware, the way it should be!

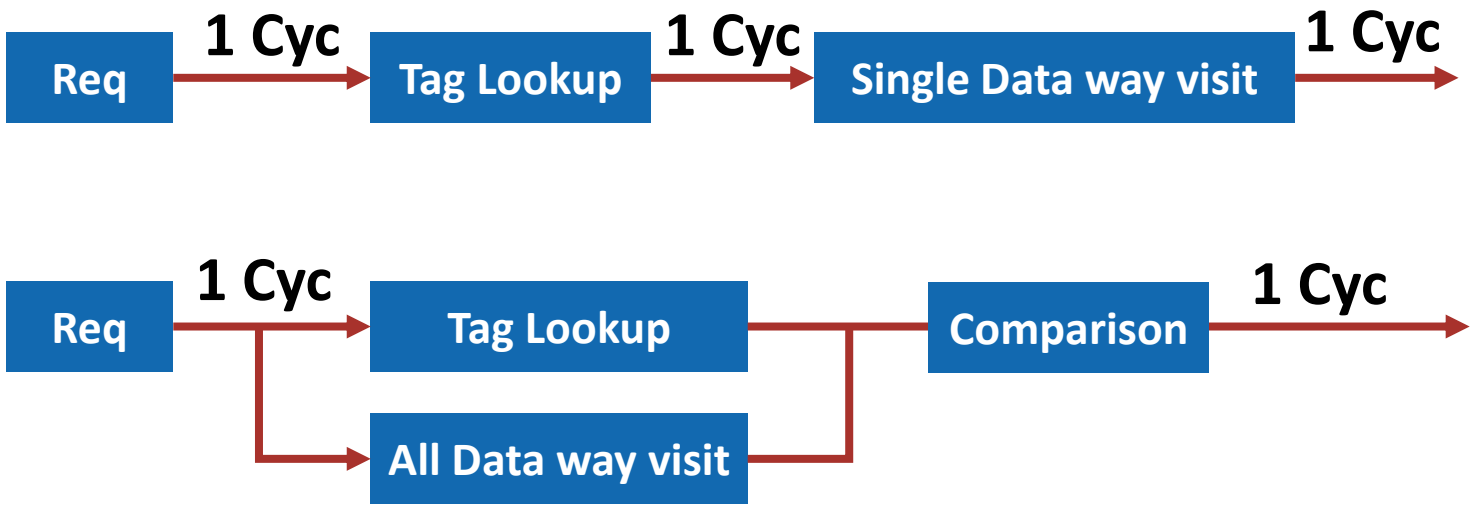
@pulp_platform
pulp-platform.org
youtube.com/pulp_platform

Hardware Development (Cluster)



Hardware Development (Cluster)

- **Cache Lookup/Replacement Policy Optimization**
  - Findings from InSitu-cache paper
  - Cache consumes 4 times higher power than expected
    - Simultaneous lookup all ways upon visiting

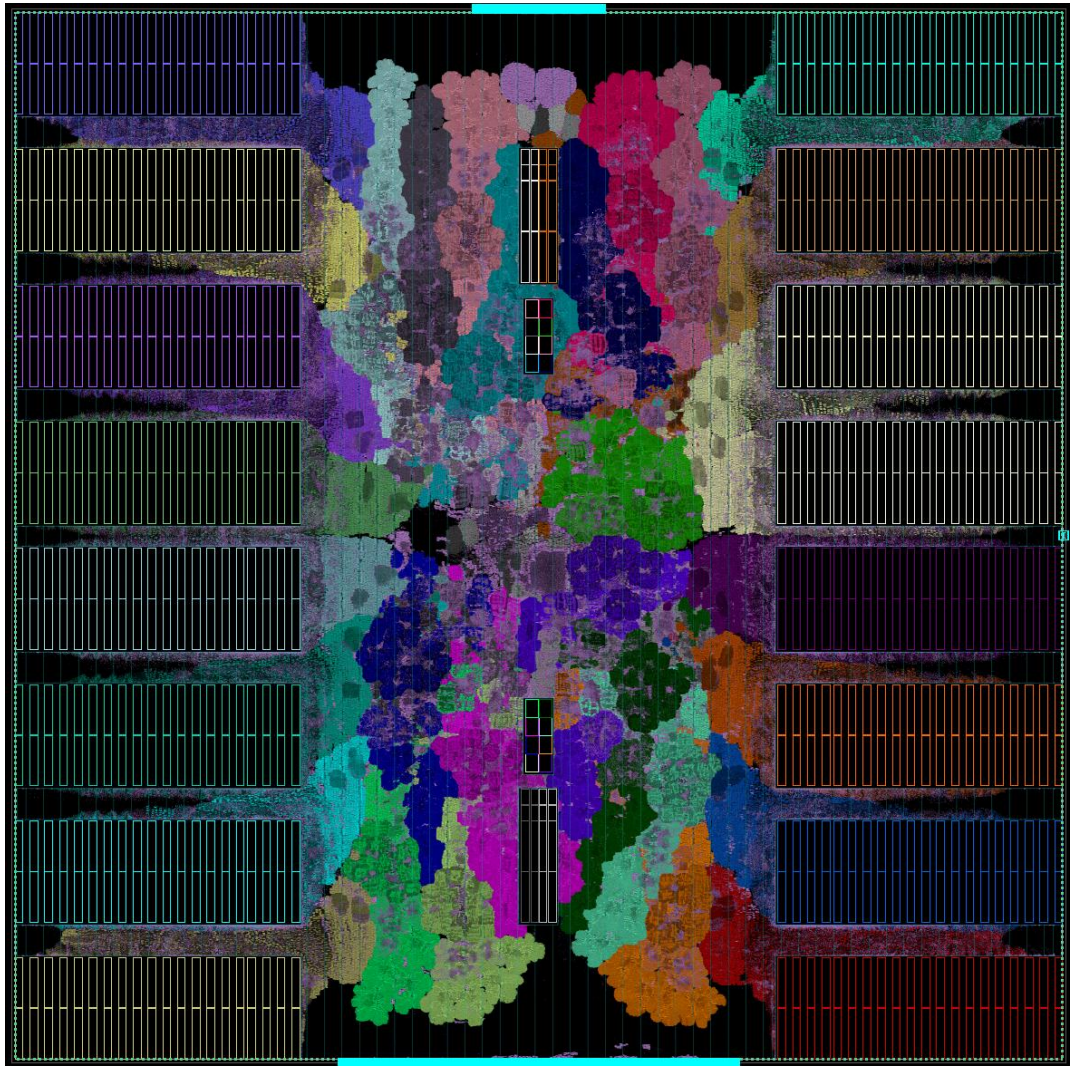


Hardware Development (Cluster)

- **Cache Lookup/Replacement Policy Optimization**
  - Findings from InSitu-cache paper
  - Cache consumes 4 times higher power than expected
    - Simultaneous lookup all ways upon visiting
  - A simple solution without latency punishment
    - Deterministic way mapping
    - One data always map to a way => we know which way needs to lookup
    - Cost: may underutilize cache resources

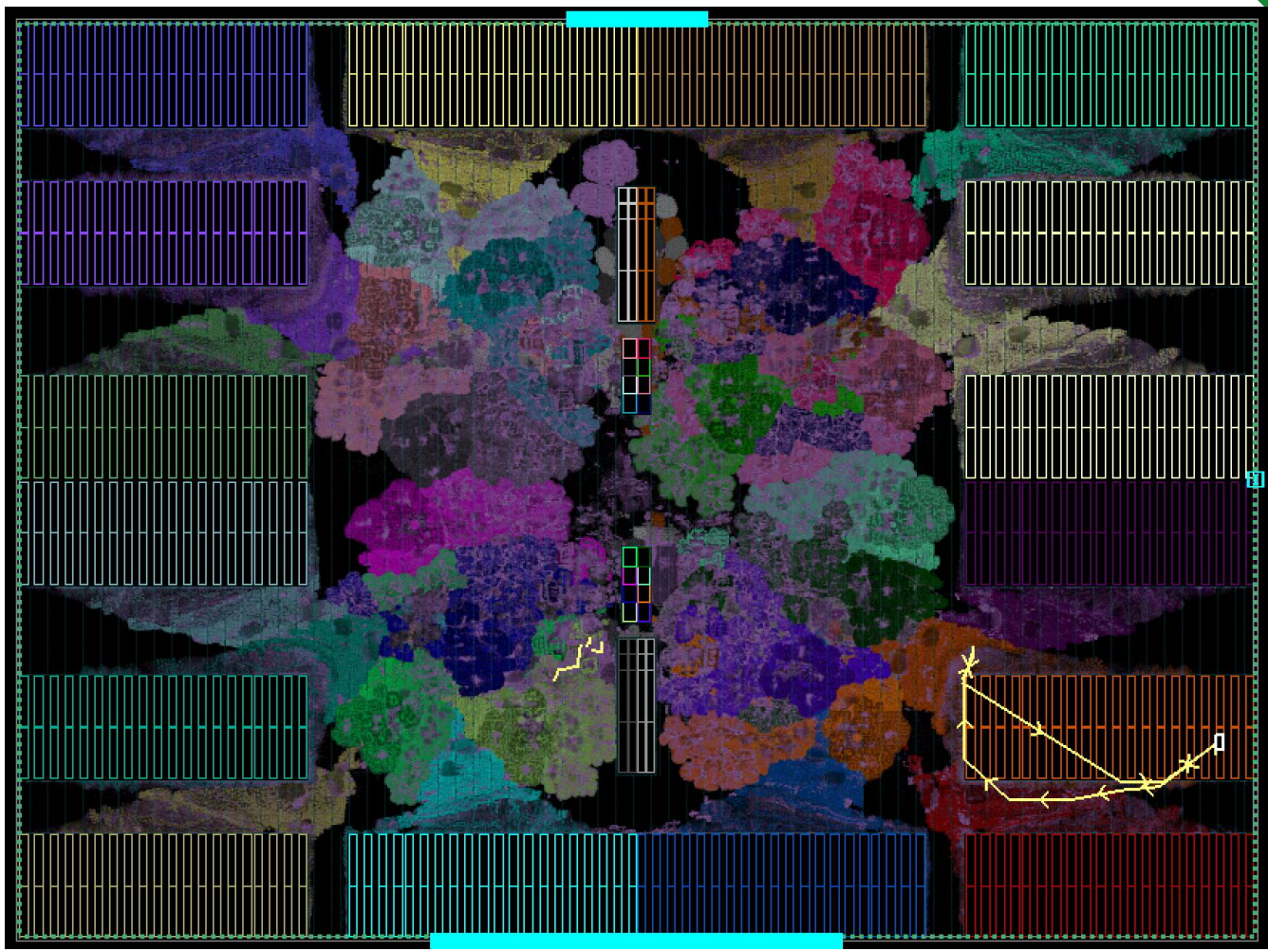
Hardware Development (Cluster)

- **Backend**
  - Adjust floorplan
  - Same die size (9 mm2, 50% utilization)
  - Timing improved, but still with 70 ps WNS



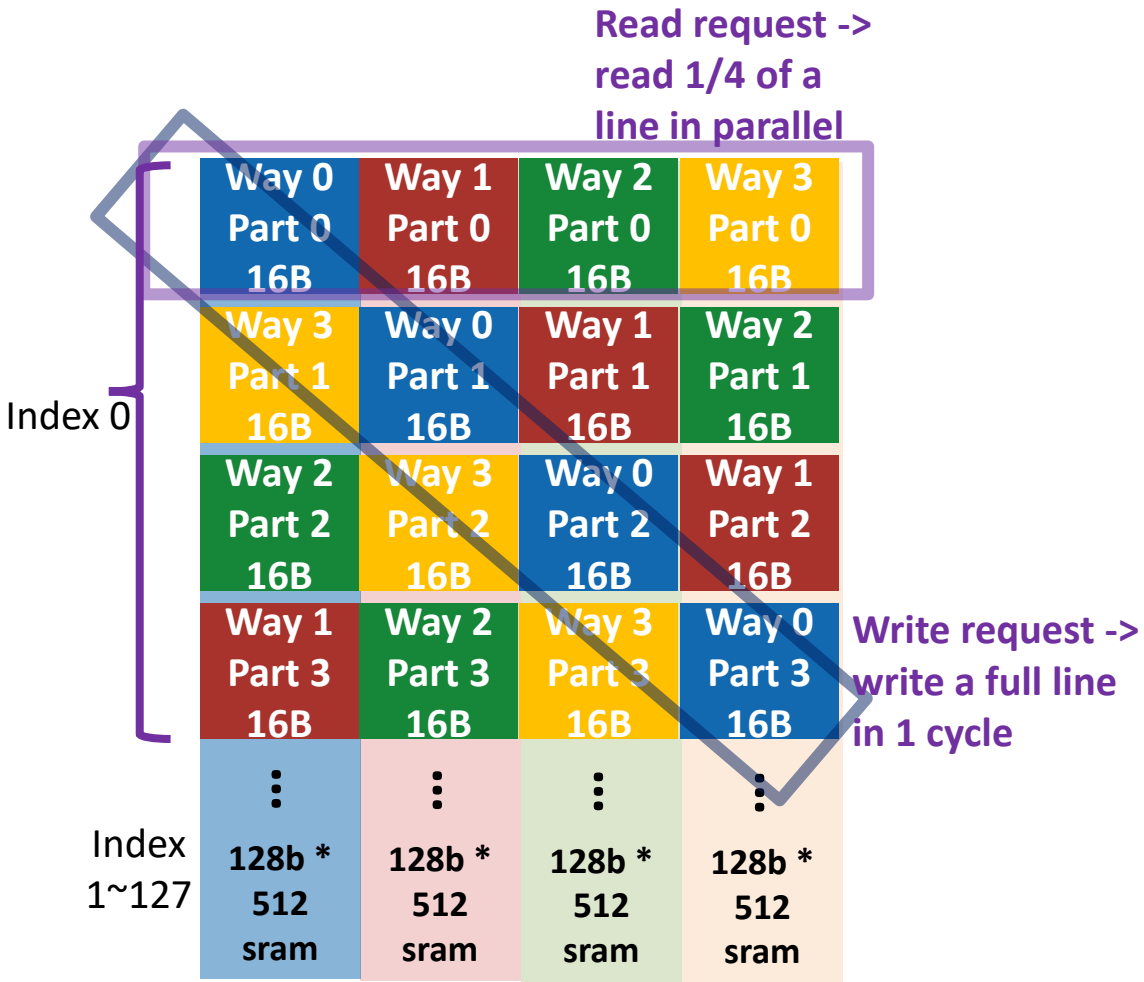
Hardware Development (Cluster)

- **Backend**
  - Adjust floorplan
  - Same die size (9 mm2, 50% utilization)
  - Timing improved, but still with 70 ps WNS
- **Need folded cache to fully resolve the issue**
  - More discussions in next slides



Hardware Development (L1 Cache)

- **Progress**
  - Passed all kernels including RLC kernel
- **Issue**
  - Unexpected performance loss
    - ~20% more cycles for 100 RLC pkgs
  - The vector continuous load and store can only achieve 50% bandwidth compared to the previous design



Folded Cache in CachePool

- **The read hit flow of insitu cache ctrl**
  - Cycle 1: Preread SRAM
    - read meta SRAM (need LRU + dirty + tag)
    - read data SRAM for all ways data
  - Cycle 2: FSM + Write SRAM
    - Generate new LRU
    - write meta SRAM (updated LRU)
  - Cycle 3: Next preread can issue

For continuous read hit to the same cache line (512b), 2 cycles per request

Folded Cache in CachePool

- **The write hit flow of insitu cache ctrl**
  - Cycle 1: Preread SRAM
    - read meta SRAM (need cache_mask + dirty + LRU + tag)
    - read data SRAM for all ways
  - Cycle 2: FSM + Write SRAM
    - generate new_mask byte dirty mask
    - generate new LRU
    - merge new bytes into old line
    - write meta SRAM (updated dirty mask + dirty + LRU)
    - write data SRAM
  - Cycle 3: Next preread can issue

For continuous write hit to the same cache line (512b), 2 cycles per request

Folded Cache in CachePool

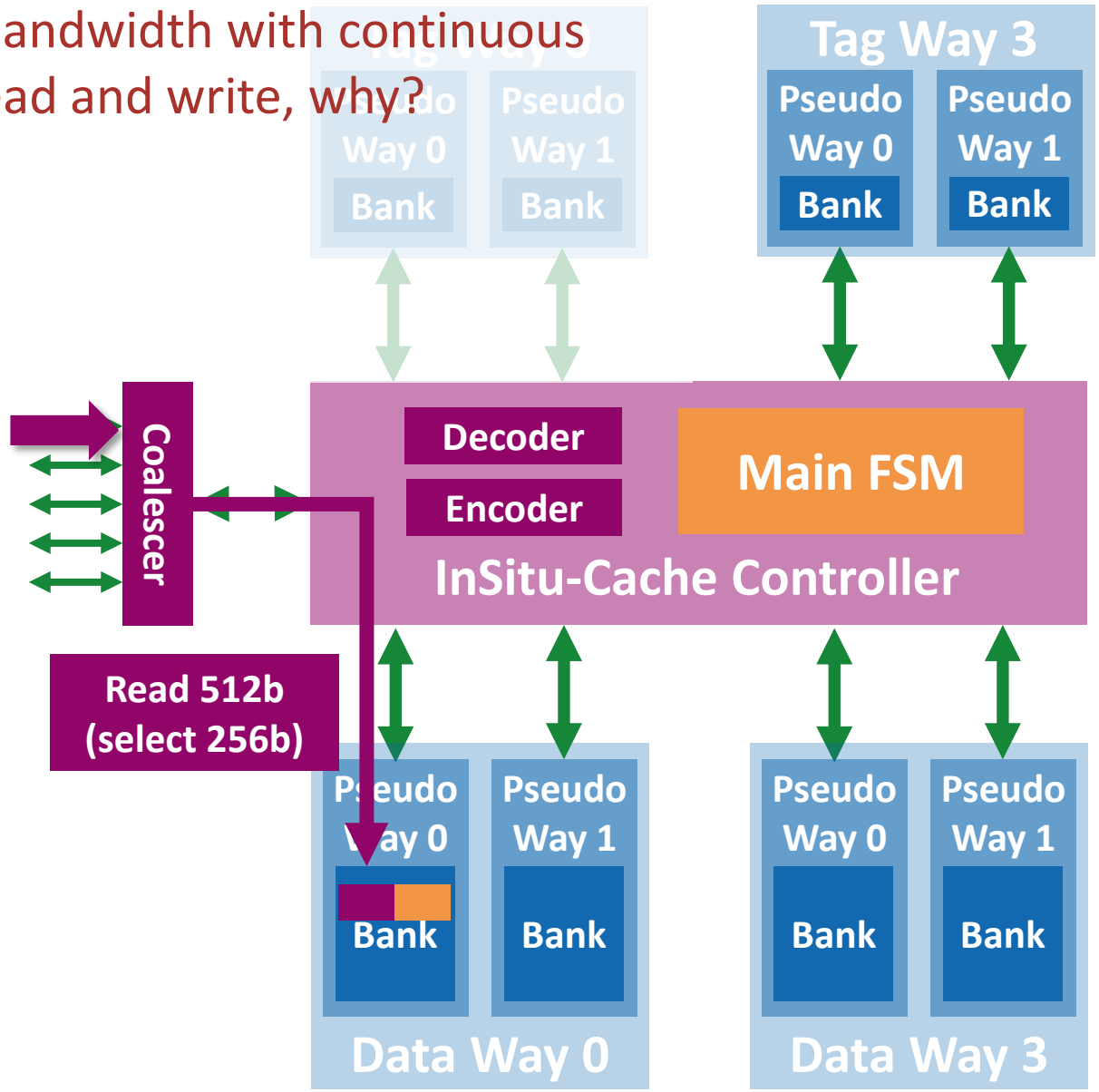
- **The cache miss flow of insitu cache ctrl**
  - Cycle 1: Preread SRAM
    - read meta SRAM
    - read data SRAM for all ways
  - Cycle 2: FSM + Write SRAM
    - generate new mshr
    - generate new dirty mask, dirty, LRU
    - write meta SRAM (updated dirty mask + dirty + LRU)
    - write data SRAM (update MSHR)
  - Cycle 3: Next preread can issue

For continuous miss to the same cache line (512b), 2 cycles per request

Previous Design

But the previous design can still sustain the full bandwidth with continuous vector read and write, why?

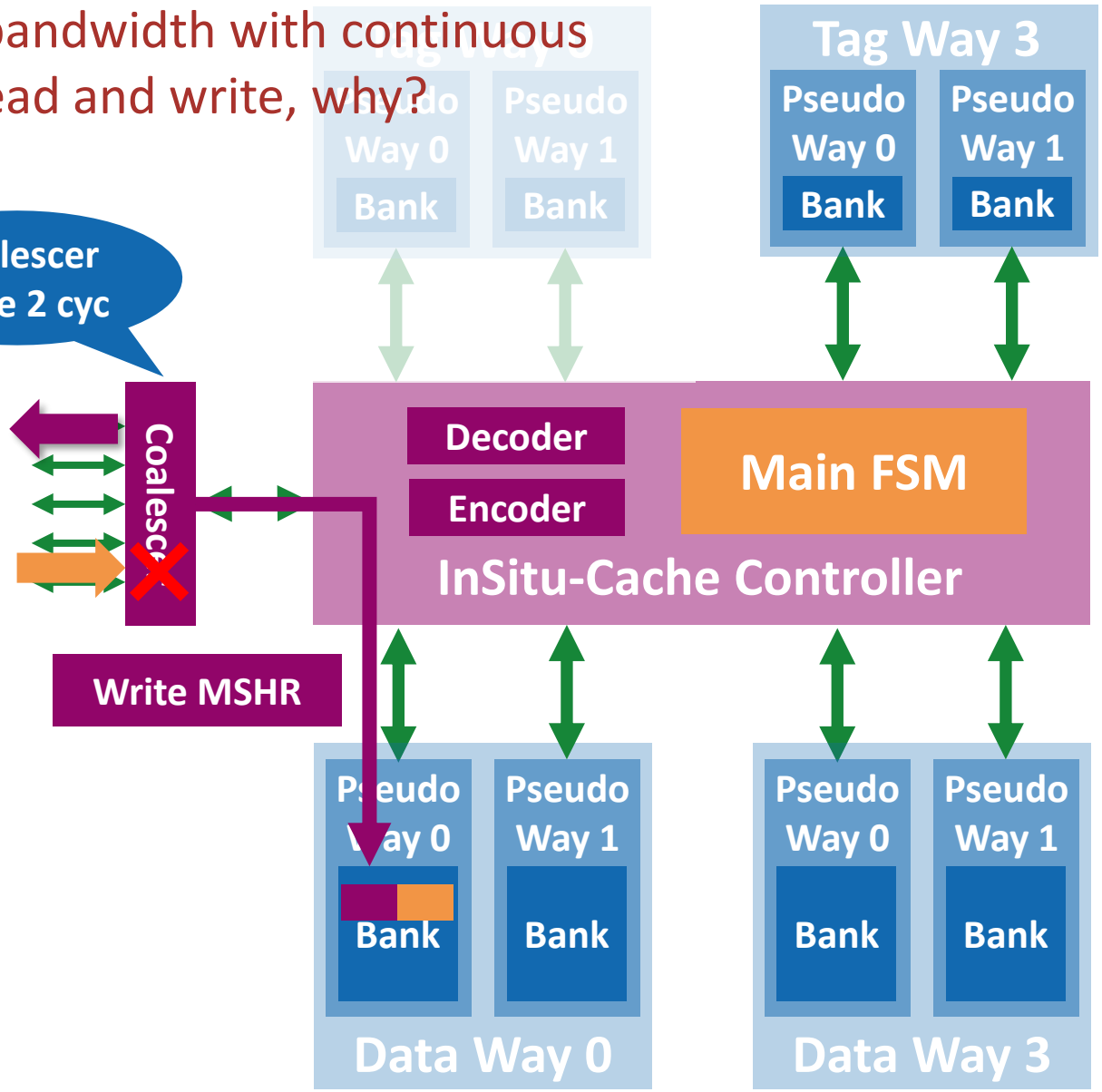
- **Granularity Mismatch**
  - Cacheline: 512b, match with DRAM
    - Each access gets a 512b data, select 256b out of it
  - Core: 128b (4*32bit)
- **Previous Design**
  - Use Coalescer to absorb mismatch
  - Every **2 cycles** form a 256b visit
  - Read 512b with 2 visits from same entry
  - Coalescer needs **2 cycles** to output one **256b** request, which hides the conflict cycle



Previous Design

But the previous design can still sustain the full bandwidth with continuous vector read and write, why?

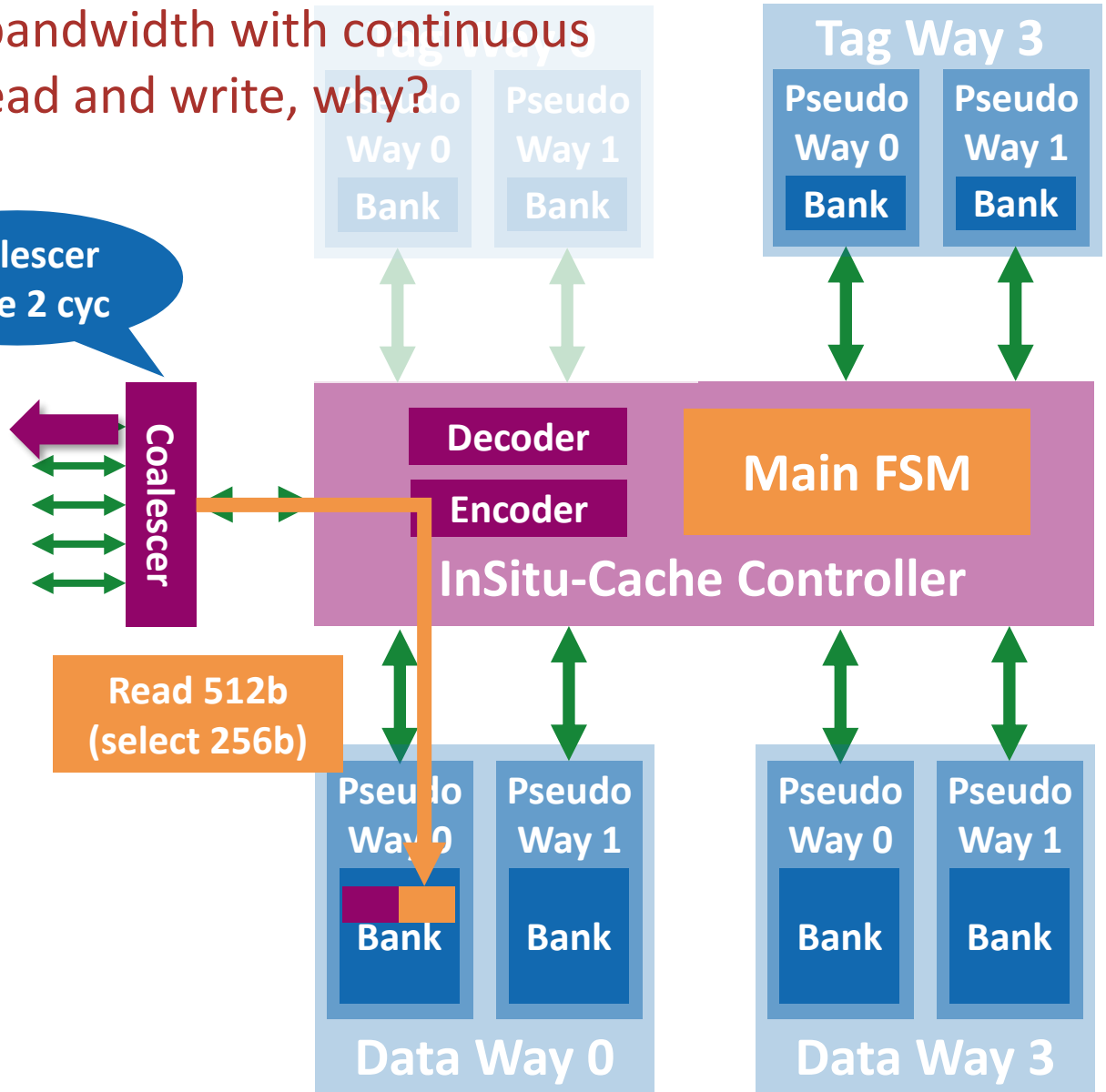
- **Granularity Mismatch**
  - Cacheline: 512b, match with DRAM
    - Each access gets a 512b data, select 256b out of it
  - Core: 128b (4*32bit)
- **Previous Design**
  - Use Coalescer to absorb mismatch
  - Every **2 cycles** form a 256b visit
  - Read 512b with 2 visits from same entry
  - Coalescer needs **2 cycles** to output one **256b** request, which hides the conflict cycle



Previous Design

But the previous design can still sustain the full bandwidth with continuous vector read and write, why?

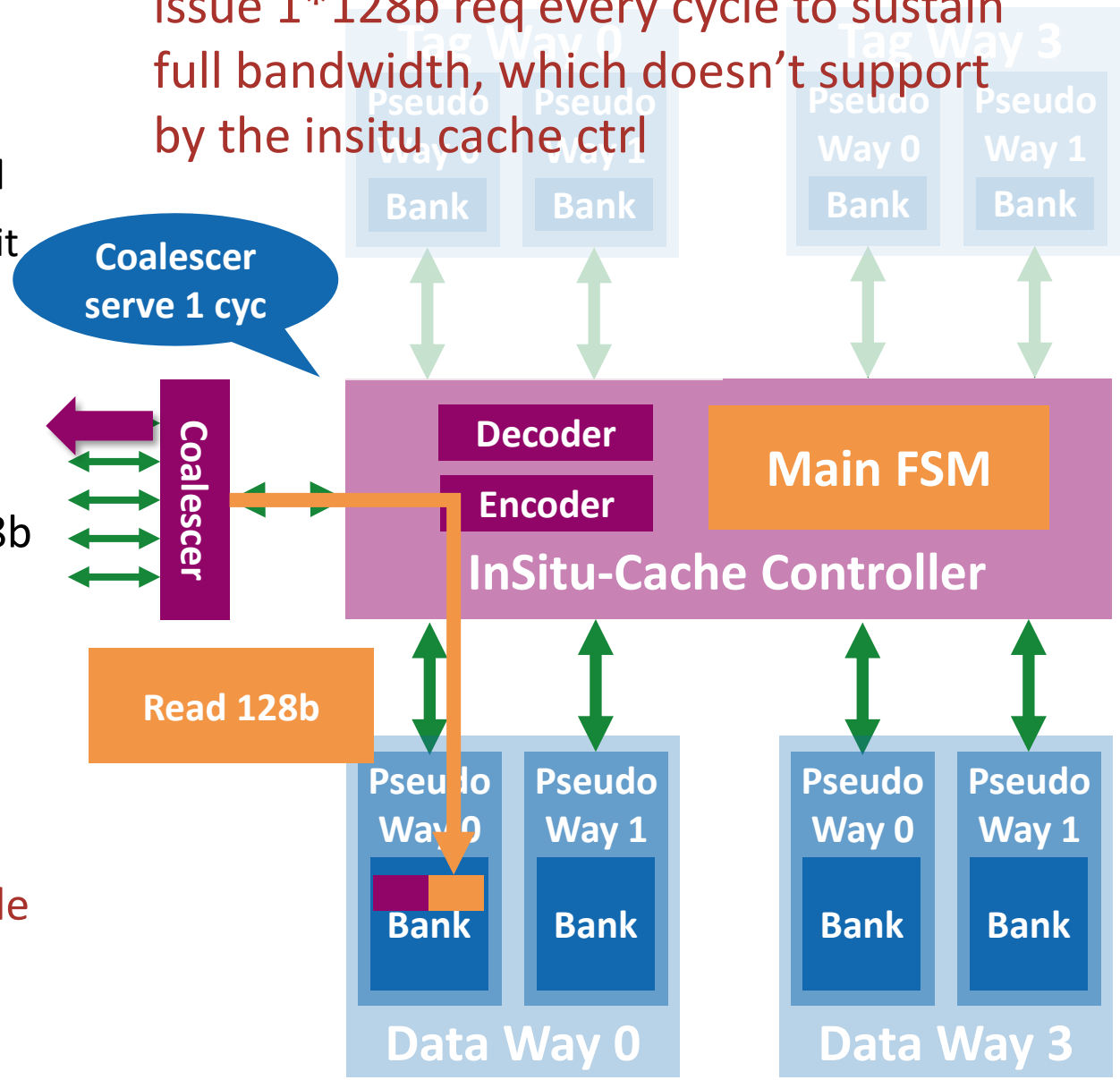
- **Granularity Mismatch**
  - Cacheline: 512b, match with DRAM
    - Each access gets a 512b data, select 256b out of it
  - Core: 128b (4*32bit)
- **Previous Design**
  - Use Coalescer to absorb mismatch
  - Every **2 cycles** form a 256b visit
  - Read 512b with 2 visits from same entry
  - Coalescer needs **2 cycles** to output one **256b** request, which hides the conflict cycle



Folded Cache

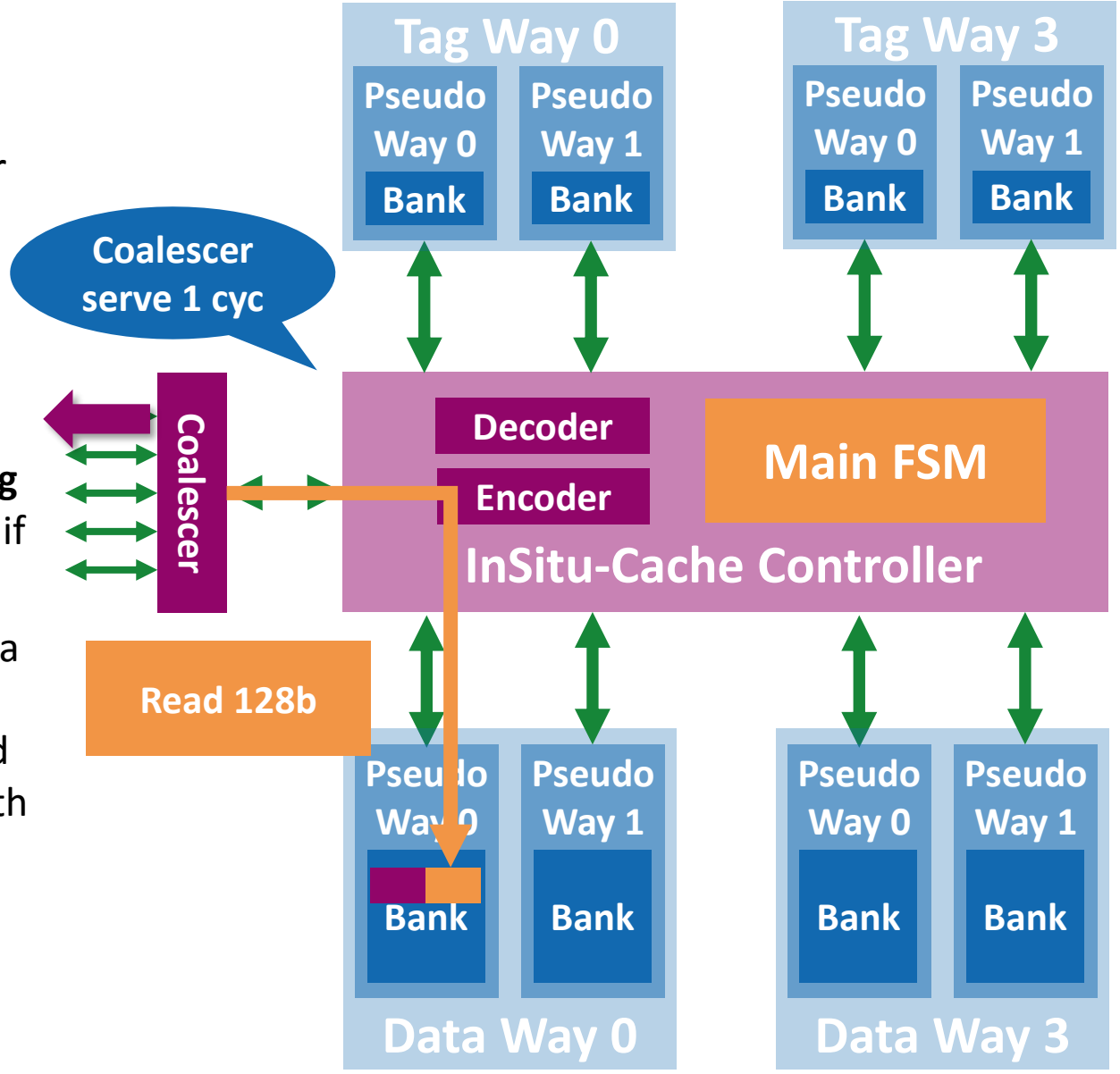
- **Granularity Mismatch**
  - Cacheline: 512b, match with DRAM
    - Folded cache: Each access gets 128bit part
  - Core: 128b (4*32bit)
- **Folded Cache Design**
  - Every 1 cycles Coalescer form a 128b visit → 1 req to **cache ctrl** from coalescer **every cycle**
  - Read 512b with 4 visits from same entry
  - Coalescer needs **1 cycles** to output one **128b** requests, the conflict cycle can't be hided

With folded design, the coalescer has to issue 1*128b req every cycle to sustain full bandwidth, which doesn't support by the insitu cache ctrl



What can we do with the folded cache?

- **Plan**
  - Use "**Pseudo-Random**", save power and get rid of the **LRU** read/write
  - Add a register based **forwarding buffer**, which can buffer the last read from/write to the SRAM bank.
    - If the SRAM access hit the **forwarding buffer**, no need to access the SRAM, if miss, we **write back** to SRAM
    - We have **2 pseudo ways** for each data way, so likely the request to the next line goes to another pseudo way, and the buffer **write back** can overlap with the new SRAM access.



Next Step

- **InSitu-Cache Paper**
  - Both Zexin and Diyou will focus on the InSitu cache paper in March. The progress on other side might be delayed
- **Diyou Side**
  - Finish cache partitioning
  - Finish backend exploration
  - Merge folded cache, collect benchmark and backend results
- **Zexin Side**
  - Fix the bandwidth issue introduced by folded cache design
  - Start the backend evaluation of the new cache controller design

Timeline

		2025					2026							
		11	12	1	2	3	4	5	6	7	8	9	10	11
System	Multi-Tile Scaling [RTL]													
	4/16-Group Scaling [GVSoC + RTL]													
L1 Access Latency	Hardware optimization (Cache & Interco) [RTL]													
	Cache partitioning [RTL]													
	WT-L1 by master student [RTL]													
Large NoC latency	Reduce per-hop latency [Physical]													
	Cut clk-domain (optional) [RTL + Physical]													
	NoC Topologies [RTL + GVSoC + Physical]													

Timeline

		2025					2026							
		11	12	1	2	3	4	5	6	7	8	9	10	11
Improve scalar core performance	Shared vector core [RTL + GVSoC]													
SW Kernel	Kernel optimization [SW]													
	Kernel development [SW]													
Report	Final Report and Benchmark													

L1 Access Latency

- **Problem: increased L1 access latency when scaling up**
- **Solutions**
  - **(Ongoing, RTL & Physical) Hardware optimization**
    Further optimize the interconnection and cache controller to reduce access latency on existing architecture
    => Retiming the current interconnection, remove unnecessary cuts in interco and cache
    => Target to reduce 1-2 cycles (~5 cyc) for hit-to-use latency
  - **(Ongoing, RTL) Cache partitioning**
    Partition the shared L1 to keep data localized to avoid remote access latency
  - **(Ongoing, RTL & Physical) WT-L1**
    Explore and evaluate a private WT L1 for each core
    => Ongoing master thesis, now is working on the coherence between L1 caches
    => Currently plan to use hierarchical directory-based approach

L1 Access Latency

- **Problem: large NoC latency (~28 cyc based on previous exploration)**
- **Solutions**
  - **(Ongoing, Physical) Reduce hop latency**
    Explore the timing under 7nm technology to see the possibility of reducing per-hop latency to 1 cycle
    => Reduce the latency by 1/2
  - **(Planned, Physical) Cut clk-domain (optional)**
    If 1-cyc is not possible, explore to operate the NoC under a higher frequency than cores (e.g. 1.5GHz)
    => Effectively cut the latency by 1/3
  - **(Planned, GVSoC & Physical) NoC Topology**
    Explore different NoC topologies
    => Torus to reduce the diameter of NoC by 1/2, reduce longest latency by 1/2

PE-Related

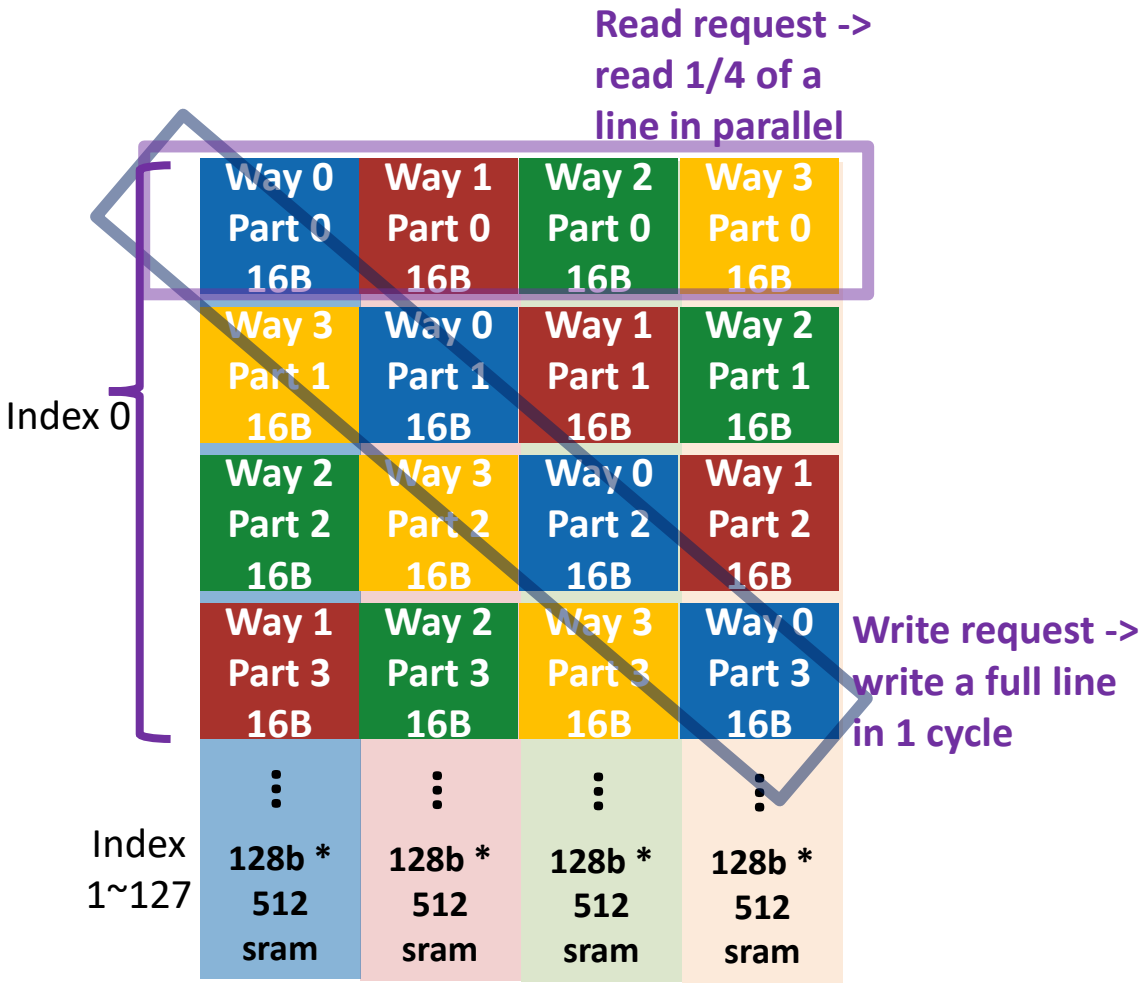
- **Problem: Improve scalar core performance**
- **Solutions**
  - **(Planned, RTL & GVSoC) Shared vector core**
    Explore two Snitch sharing one Spatz configuration to improve scalar performance
    Software complexity needs to be considered, especially on the possible conflicting use of register file.

RLC Workload

- **Problem: Current RLC model cannot reveal all properties of the kernel**
- **Solutions**
  - **(Ongoing, SW) Kernel optimization**
    Adapt the current model to better represent the real cases, e.g. do some operations on the data. Further cooperation with HQ side to develop a more complete and better model.
  - **(Planned, SW) Kernel development**
    Implement the missing use cases for multi-user

Hardware Development (L1 Cache)

- **Progress**
  - Passed all kernels including RLC kernel
- **Benefits**
  - Significant area efficiency improvement in backend
    Original: One Way: 128x512b SRAM (not generatable) => 4 128x128b SRAMs. Need 512b data, extra address and control wires
    Folded: One Way: 512x128b SRAM. 4 times less SRAM macros. 128b data, no extra wires needed



Thank you!